

SigrikaGo System Design

本文档基于当前代码库静态分析生成，目标是方便后续维护者和 AI 继续接手。未在代码中确认的内容会标注为“待确认”。

1. 项目目录结构说明

SigrikaGo/	
.env.example	# 本地环境变量模板
index.html	# Vite 入口 HTML
package.json	# npm 脚本与依赖
vite.config.js	# Vite + React 配置，开发期代理 /api 与 /uploads 到后端
docs/	
system-design.md	# 本文档
superpowers/	# 既有设计/计划文档
prisma/	
schema.prisma	# Prisma SQLite 数据模型
public/	
assets/	# 角色/其他静态图片资源
server/	
index.js	# Express + Socket.IO 服务入口
rooms.js	# 实时房间、匹配、对局流程、计时、棋谱保存
auth.js	# HTTP JWT 鉴权与管理员中间件
socketAuth.js	# Socket.IO JWT 鉴权与角色解析
adminConfig.js	# 管理员用户名配置与角色提升
adminRoutes.js	# /api/admin 后台管理路由
characters.js	# 角色/技能校验、序列化、内置角色种子
characterSelection.js	# 用户出战角色解析与 fallback
db.js	# Prisma Client 与 publicUser 序列化
shop.js	# 商城/装饰校验、价格、购买逻辑
leaderboard.js	# 排行榜统计逻辑
skillRegistry.js	# 角色技能 fallback 配置转换
*.test.js	# Vitest 单元测试
src/	
main.jsx	# React 单页应用、主要组件与前端业务逻辑
styles.css	# 全局样式
shared/	
game.js	# 13 路围棋规则、技能、数子、回放核心逻辑
characters.js	# 前端角色合并逻辑
characterFallback.js	# 内置角色 fallback 配置
*.test.js	# 共享逻辑测试

运行相关文件：

- `.env` 未纳入 Git，当前模板包含 `DATABASE_URL`、`JWT_SECRET`、`PORT`。
- `.gitignore` 排除了 `node_modules`、`dist`、`.env`、SQLite 数据库、开发日志等。
- 当前目录不是 Git 仓库本身，至少在本机环境下 `git status` 不可用。

2. 当前核心模块

前端

- `src/main.jsx`
 - React SPA 的集中入口。
 - 包含认证、大厅、对局、棋舍、商城、排行榜、设置、后台管理等组件。
 - 使用 `useState` / `useEffect` 做本地状态管理。
 - 通过 `fetch` 调用 HTTP API，通过 `socket.io-client` 连接实时对局。
- `src/shared/game.js`
 - 共享的游戏规则引擎。
 - 负责棋盘状态、落子、提子、禁自杀、劫、弃手、认输、技能、隐藏手、死子标记、数子、回放重算等。
 - 该模块被前端回放逻辑与服务端房间逻辑共同使用。
- `src/shared/characters.js` 与 `src/shared/characterFallback.js`
 - 定义内置角色 fallback。
 - 将后端 DB 角色与内置角色合并。
- `src/styles.css`
 - 单文件全局样式，覆盖全部页面、弹窗、棋盘、后台、商城等。

后端

- `server/index.js`
 - Express HTTP API 与 Socket.IO 入口。
 - 初始化内置角色 seed、提升配置管理员。
 - 注册公开 API、登录/注册 API、用户 API、棋谱 API、排行榜 API，并挂载 `/api/admin`。
- `server/rooms.js`
 - 内存态房间系统。
 - 管理匹配队列、房间成员、观战、Socket 广播、聊天、计时、读秒、和棋/数子流程、技能演出、结束后保存棋谱与更新胜负积分。
- `server/adminRoutes.js`
 - 后台管理路由。
 - 管理用户、封禁/解封、重置密码、角色、技能、装饰、商城商品、上传、审计日志、任意用户棋谱查看。
- `server/characters.js`

- 角色输入校验、技能校验、公开 payload 转换、内置角色初始化。
- `server/shop.js`
 - 商城商品和装饰校验、折扣价格计算、购买事务。
- `server/leaderboard.js`
 - 从用户与棋谱记录中统计排行榜。
- `server/auth.js` / `server/socketAuth.js`
 - HTTP 与 Socket.IO 的 JWT 鉴权。
- `server/db.js`
 - Prisma Client 单例和用户公开字段序列化。

3. 已实现功能列表

账号与用户

- 用户注册与登录。
- 密码使用 `bcryptjs` 哈希存储。
- JWT 登录态, token 有效期为 14 天。
- 用户状态支持 `active` / `banned`。
- 管理员可配置、可在启动或登录时自动提升。
- 用户公开字段序列化会隐藏 `passwordHash`。

大厅与个人空间

- 大厅入口：棋舍、空想对局、观战、排行榜、商城、后台管理（管理员可见）。
- 棋舍展示用户战绩、积分、拥有角色、出战角色。
- 棋舍可查看个人对局回放。
- 设置弹窗支持音量配置，并保存在 `localStorage`。

对局

- Socket.IO 两人匹配。
- 5 位房间号观战。
- 13 路棋盘。
- 中国数子规则, `KOMI_STONES = 2.75`。
- 基础规则：落子、提子、禁自杀、劫、弃手、认输。
- 对局聊天和系统消息。
- 计时与读秒：房间玩家包含 `mainTime`、`byoYomi`、`byoYomiPeriods` 等内存字段。
- 超时判负。
- 双方连续弃手进入数子申请/确认流程。
- 数子流程：申请、接受/拒绝、标记死子、标记单官/中立点、确认死子、结果确认、拒绝后继续。

- 和棋申请，10 秒超时拒绝。
- 结束后保存棋谱，并更新胜者积分/胜局、败者负局。

技能与角色

- 内置角色 fallback: `sigrika`、`danea`、`aemeath`。
- DB 角色会覆盖/合并内置角色。
- 技能类型：
 - `erase-point`：抹除空交叉点，点位不可落子且不参与数子。
 - `flip-stone`：反转目标棋子颜色。
 - `hidden-hand`：隐藏手，未暴露前对对方隐藏。
- 技能可配置：
 - 使用次数 `uses`
 - 是否不消耗回合 `freeTurn`
 - 目标规则 `targetRule`
 - 参数 JSON `paramsJson`
 - 代价类型 `costType`
 - 代价值 `costValue`
 - 系统消息模板 `systemMessage`
- 系统消息模板当前支持：`{player}`、`{character}`、`{skill}`、`{point}`、`{fromColor}`、`{toColor}`、`{targetColor}`，并保留 `{color}` 兼容旧配置。

棋谱与回放

- 对局结束后写入 `GameRecord`。
- 棋谱保存房间快照 `snapshot`。
- 用户可看自己的最近 30 条棋谱。
- 管理员可查看任意用户棋谱与任意棋谱详情。
- 前端回放通过 `replayRoomAt` 基于历史步骤重放共享规则逻辑。

商城与装饰

- 商品支持 `character` 与 `decoration` 两类。
- 商品有原价、折扣、是否可购买、是否展示、排序、描述、图片。
- 购买会扣除用户金币，并写入 `ownedCharacters` 或 `ownedDecorations`。
- 装饰数据模型和后台管理已实现；玩家端装饰实际使用方式待确认。

排行榜

- 入口在大厅。
- API 为 `GET /api/leaderboard`。
- 仅展示至少完成过一盘对局的用户。
- 按 `rating` 降序排序。

- 从棋谱统计总对局数、胜局数、负局数。
- “常用角色”按棋谱中该用户使用次数最多的角色计算。

后台管理

- 概览：用户数、封禁用户数、启用角色数、棋谱数、最近审计日志。
- 用户管理：列表、编辑角色/段位/积分/金币/拥有角色/出战角色、封禁、解封、重置密码、查看用户棋谱。
- 角色管理：新增/编辑/禁用角色，配置技能系统。
- 角色立绘上传：支持 png/jpeg/webp/gif，大小限制 3MB。
- 商城管理：增改商品、下架商品。
- 装饰管理：增改装饰、停用装饰。
- 审计日志：记录部分后台操作前后值。

4. 数据模型与字段说明

数据源定义在 `prisma/schema.prisma`，数据库为 SQLite。

User

用户账号与资产/战绩。

- `id`：主键 `cuid`。
- `username`：唯一用户名。
- `passwordHash`：bcrypt 哈希密码。
- `role`：用户角色，当前代码使用 `player` / `admin`。
- `status`：用户状态，当前代码使用 `active` / `banned`。
- `banReason`：封禁原因，可空。
- `bannedAt`：封禁时间，可空。
- `rank`：段位/等级文本。默认值在当前 `schema` 终端显示存在编码问题，语义待确认。
- `rating`：积分，默认 1000。
- `wins`：胜局数，默认 0。
- `losses`：负局数，默认 0。
- `coins`：金币，默认 300。
- `selectedCharacter`：出战角色 slug，默认 `sigrika`。
- `ownedCharacters`：逗号分隔的角色 slug 字符串，默认包含内置角色。
- `ownedItems`：逗号分隔字符串；当前代码未看到具体购买/使用逻辑，待确认。
- `ownedDecorations`：逗号分隔装饰 slug。
- `createdAt`，`updatedAt`：创建和更新时间。

GameRecord

结束对局记录。

- `id`：主键 `cuid`。

- `roomCode` : 房间号。
- `blackUserId` , `whiteUserId` : 黑白双方用户 id。
- `blackName` , `whiteName` : 保存时的用户名快照。
- `blackCharacter` , `whiteCharacter` : 双方角色 slug 快照。
- `resultText` : 结果文本。
- `moveCount` : 手数。
- `snapshot` : JSON 字符串, 保存 `roomView` 快照。
- `createdAt` : 创建时间。

Character

可配置角色。

- `id` : 主键 cuid。
- `slug` : 唯一角色标识, 用于前端匹配、出战角色、拥有角色、棋谱角色字段。
- `name` : 角色名。
- `portraitUrl` : 立绘 URL 或静态路径。
- `portraitSource` : url 或 upload 。
- `palette` : 代表色。
- `enabled` : 是否启用。
- `sortOrder` : 排序。
- `skill` : 一对一 `CharacterSkill` 。
- `createdAt` , `updatedAt` : 创建和更新时间。

CharacterSkill

角色技能配置。

- `id` : 主键 cuid。
- `characterId` : 关联 `Character.id` , 唯一。
- `effectType` : 技能实际效果类型, 当前支持 `erase-point` 、 `flip-stone` 、 `hidden-hand` 。
- `name` : 技能名。
- `description` : 技能描述。
- `uses` : 每局使用次数。
- `freeTurn` : 是否不消耗回合。
- `targetRule` : 目标规则, 当前校验为 `empty-point` 或 `stone` 。
- `paramsJson` : JSON 字符串, 当前作为扩展参数保留。
- `costType` : `numeric` 或 `special` 。
- `costValue` : 代价值; `numeric` 会参与数子扣分, `special` 当前仅展示。
- `systemMessage` : 技能系统消息模板。
- `enabled` : 是否启用; 当前公开角色 payload 未看到按 `skill.enabled` 过滤, 待确认。
- `createdAt` , `updatedAt` : 创建和更新时间。

Decoration

装饰配置。

- `id` : 主键 `cuid`。
- `slug` : 唯一装饰标识。
- `name` : 装饰名。
- `description` : 描述。
- `imageUrl` : 图片地址。
- `enabled` : 是否启用。
- `sortOrder` : 排序。
- `createdAt` , `updatedAt` : 创建和更新时间。

ShopItem

商城商品。

- `id` : 主键 `cuid`。
- `name` : 商品名。
- `category` : `character` 或 `decoration` 。
- `targetId` : 商品对应角色 `slug` 或装饰 `slug`。
- `priceCoins` : 原价金币。
- `discountPercent` : 折扣百分比 0-100。
- `purchasable` : 是否可购买。
- `enabled` : 是否展示。
- `sortOrder` : 排序。
- `description` : 描述。
- `imageUrl` : 图片地址。
- `createdAt` , `updatedAt` : 创建和更新时间。

AdminAuditLog

后台审计日志。

- `id` : 主键 `cuid`。
- `adminUserId` : 操作管理员用户 `id`。
- `action` : 操作类型, 如 `user.update` 、 `character.update` 。
- `targetType` : 目标类型, 如 `user` 、 `character` 。
- `targetId` : 目标 `id` 或 `slug`。
- `beforeJson` : 操作前 JSON, 可空。
- `afterJson` : 操作后 JSON, 可空。
- `createdAt` : 创建时间。

5. 数据之间的关系

- `Character` 与 `CharacterSkill` 是一对一关系：
 - `CharacterSkill.characterId` 唯一并引用 `Character.id`。
 - 删除角色会级联删除技能。
- `User` 与 `GameRecord` 没有 Prisma relation 声明，但通过字符串字段形成逻辑关系：
 - `GameRecord.blackUserId` / `whiteUserId` 对应 `User.id`。
 - `GameRecord.blackName` / `whiteName` 是用户名快照。
- `Character` 与 `GameRecord` 没有 Prisma relation 声明：
 - `GameRecord.blackCharacter` / `whiteCharacter` 保存角色 slug 快照。
 - 前端展示时通过 `findCharacter` 在当前角色集合中解析。
- `User` 与角色/装饰拥有关系没有 join table：
 - `User.ownedCharacters`、`ownedDecorations` 使用逗号分隔字符串。
 - `User.selectedCharacter` 保存单个角色 slug。
- `ShopItem.targetId` 与角色/装饰没有数据库外键：
 - `category = character` 时逻辑上指向 `Character.slug`。
 - `category = decoration` 时逻辑上指向 `Decoration.slug`。
- 实时 `Room`、`Player`、`GameState` 不是数据库模型：
 - 保存在 `server/rooms.js` 的内存 `Map`。
 - 对局结束后才以 `GameRecord.snapshot` 形式落库。

6. API 路由与接口用途

公开/普通用户 HTTP API

- `GET /api/health`
 - 健康检查。
- `GET /api/characters`
 - 返回启用角色列表和停用 slug 列表。
 - 不要求登录。
- `POST /api/auth/register`
 - 注册账号。
 - 用户名至少 2 位，密码至少 4 位。
- `POST /api/auth/login`
 - 登录并返回 JWT 与公开用户信息。
 - 被封禁用户返回 403。
- `GET /api/me`
 - 登录用户信息。

- 会基于棋谱重新计算展示用 `wins`、`losses`、`rating = 1000 + wins * 20`。
- `POST /api/me/character`
 - 修改当前出战角色。
 - 会校验角色存在且未停用；逻辑同时考虑 DB 角色和 fallback 角色。
- `GET /api/shop`
 - 登录后获取启用商城商品。
- `POST /api/shop/:id/purchase`
 - 登录后购买商品。
 - 扣金币并更新拥有角色/装饰。
- `GET /api/leaderboard`
 - 登录后获取排行榜。
 - 基于所有用户和所有棋谱统计。
- `GET /api/replays`
 - 获取当前用户最近 30 条棋谱。
- `GET /api/replays/:id`
 - 获取当前用户参与的指定棋谱详情与快照。

后台 HTTP API

所有 `/api/admin/*` 路由都经过 `authHttp` 和 `requireAdmin`。

- `GET /api/admin/summary`
 - 后台概览数据和最近 8 条审计日志。
- `GET /api/admin/audit-logs`
 - 最近 100 条审计日志。
- `GET /api/admin/users`
 - 最多 200 个用户。
- `GET /api/admin/users/:id`
 - 单个用户详情。
- `GET /api/admin/users/:id/replays`
 - 任意用户最多 100 条棋谱。
- `GET /api/admin/replays/:id`
 - 任意棋谱详情。
- `PATCH /api/admin/users/:id`
 - 编辑用户 `role`、`rank`、`rating`、`coins`、`ownedCharacters`、`selectedCharacter`。
- `POST /api/admin/users/:id/ban`

- 封禁用户。
- `POST /api/admin/users/:id/unban`
 - 解封用户。
- `POST /api/admin/users/:id/reset-password`
 - 重置用户密码。
- `GET /api/admin/characters`
 - 后台角色列表，包含停用角色与技能 `paramsJson`。
- `POST /api/admin/characters`
 - 新增角色与技能。
- `PATCH /api/admin/characters/:id`
 - 按 DB id 或 slug 更新角色与技能。
- `DELETE /api/admin/characters/:id`
 - 将角色 `enabled` 设为 `false`。
- `POST /api/admin/uploads/character-portrait`
 - 上传角色立绘。
 - 需要 `multipart` 字段 `portrait`。
- `GET /api/admin/decorations`
 - 装饰列表。
- `POST /api/admin/decorations`
 - 新增装饰。
- `PATCH /api/admin/decorations/:id`
 - 更新装饰。
- `DELETE /api/admin/decorations/:id`
 - 将装饰 `enabled` 设为 `false`。
- `GET /api/admin/shop-items`
 - 商城商品列表。
- `POST /api/admin/shop-items`
 - 新增商品。
- `PATCH /api/admin/shop-items/:id`
 - 更新商品。
- `DELETE /api/admin/shop-items/:id`
 - 将商品 `enabled` 设为 `false`。

Socket.IO 事件

鉴权：

- `io.use`
 - 使用 JWT 鉴权。
 - 校验用户存在、未封禁，并解析出战角色。

服务端监听：

- `match:join`
 - 加入匹配队列；匹配成功后创建房间并广播。
- `match:leave`
 - 离开匹配队列。
- `room:join`
 - 根据 5 位房间号加入观战或重连玩家。
- `game:action`
 - 对局动作：`move`、`pass`、`resign`、`skill`。
- `counting:request`
 - 申请数子。
- `counting:respond`
 - 接受/拒绝数子。
- `draw:request`
 - 申请和棋。
- `draw:respond`
 - 接受/拒绝和棋。
- `scoring:action`
 - 数子阶段动作：标记死子、中立点、重置、确认、接受/拒绝结果。
- `chat:send`
 - 房间聊天。
- `disconnect`
 - 清理匹配队列 socket 和观战列表。

服务端发出：

- `match:waiting`
- `match:found`
- `room:update`
- `room:closed`
- `error:toast`
- `me`

7. 权限系统现状

- 登录态使用 JWT:
 - `withToken` 签发 `{ sub: user.id }`, 有效期 14 天。
 - HTTP 请求通过 `Authorization: Bearer <token>`。
 - Socket.IO 通过 `handshake.auth.token`。
- 用户角色:
 - `player`
 - `admin`
- 用户状态:
 - `active`
 - `banned`
- 管理员来源:
 - `.env` 中 `ADMIN_USERNAMES` 逗号分隔。
 - 服务启动时 `promoteConfiguredAdmins` 会批量提升已存在用户。
 - 登录/注册时 `syncConfiguredAdmin` 会提升匹配用户名。
- 后台权限:
 - `/api/admin` 路由挂载前统一使用 `authHttp` 和 `requireAdmin`。
 - 前端只对 `user.role === "admin"` 显示后台入口, 但真正权限以后端为准。
- 封禁效果:
 - HTTP 鉴权发现 `status === banned` 返回 403。
 - Socket 鉴权发现封禁抛出 `forbidden`。
 - 登录时封禁用户也返回 403。
- 当前未看到:
 - CSRF 防护。
 - 登录/注册限流。
 - JWT refresh / 主动失效机制。
 - 操作级细粒度权限。

8. 公共组件与通用逻辑

前端公共组件

当前公共组件都集中在 `src/main.jsx` :

- `Panel`: 面板标题与图标容器。
- `Stat`: 小统计卡片。
- `AdminFieldLabel`: 带 title 提示的后台字段标签。

- `Toast` : 自动消失提示。
- `ConfirmModal` : 通用确认弹窗。
- `WatchPad` : 观战房间号输入。
- `ReplayBar` : 回放进度控制。
- `PlayerInfo` : 对局双方信息。
- `Board` : 棋盘渲染与点击处理。
- `ChatBox` : 聊天面板。
- `SkillBanner` : 技能演出浮层。
- `ResultModal` : 对局结果弹窗。

前端通用函数

- `api` : HTTP JSON 请求封装。
- `adminApi` : `/api/admin` 请求封装。
- `uploadPortrait` : 上传角色立绘。
- `findCharacter` : 从角色 `map` 或 `fallback` 中解析角色。
- `deriveHouseStats` : 从个人棋谱推导棋舍展示战绩。
- `winnerColorFromRecord` : 基于结果文本判断胜方。
- `buildCharacterDraft` / `characterDraftToBody` : 后台角色表单数据转换。
- `validateShopItemDraft` / `decorationDraftToBody` : 后台商城/装饰表单校验。
- `replayRoomAt` : 用历史记录重放房间状态。
- 音频相关: `loadAudioSettings`、`playStoneSound`、`playCountdownBeep`、`speakText`。

后端通用逻辑

- `publicUser` : 用户公开字段白名单。
- `makeAuth` : HTTP 鉴权与管理员中间件。
- `validateCharacterInput` : 角色/技能输入校验。
- `toCharacterPayload` : 角色公开 `payload`。
- `validateShopItemInput` / `validateDecorationInput` : 商城与装饰校验。
- `buildLeaderboard` : 排行榜统计。
- `safeUploadFilename` : 上传文件名清洗。
- `writeAudit` : 后台审计日志写入。

9. 状态管理方式

前端状态

- 使用 `React` 本地状态, 没有 `Redux/Zustand` 等全局状态库。
- 顶层 `App` 管理:
 - `token`
 - `user`

- `view`
- `room`
- `socket`
- 各类弹窗开关
- `characters`
- `replayRecords`
- `replayStep`
- 音频配置
- 登录 token 存在 `localStorage` 的 `sigrika-token`。
- 音频设置存在 `localStorage` 的 `sigrika-audio-settings`。
- 房间状态由服务端 Socket 广播覆盖到前端 `room`。
- 回放状态通过 `replayStep` 和 `replayRoomAt` 从快照历史中派生。

后端状态

- 数据库持久化：
 - 用户、角色、技能、装饰、商城、棋谱、审计日志。
- 内存状态：
 - `server/rooms.js` 的 `rooms = new Map()`。
 - `waitingPlayer` 匹配队列。
 - 房间计时器、读秒、观战者、聊天、当前棋局状态。
- 对局结束持久化：
 - `scheduleRoomClose` 调用 `saveGameRecord`。
 - 房间 5 分钟后关闭并从内存删除。

10. 已知技术债

以下只列代码中能观察到的事项：

- 部分源码、schema、README 中的中文在当前终端显示为乱码。运行时页面部分中文可能正常，但文件编码状态需要统一确认。
- 没有 Prisma migrations 目录，当前只看到 `prisma db push` 脚本。跨机器/多人协作时数据库结构变更缺少可审计迁移历史。
- `prisma db push` 在当前机器曾出现空的 `Schema engine error`，后续需要确认是不是本机环境问题。
- 实时房间在内存中：
 - 服务重启会丢失未结束房间。
 - 不支持多进程/多实例横向扩展。
- 多处胜负判断依赖 `resultText` 文本前缀，而不是结构化字段：
 - `publicUserWithHistory`

- `deriveHouseStats`
- `leaderboard`
- 管理/排行统计未来如果改文案，可能影响统计。
- `GameRecord` 与 `User`、`Character` 没有数据库外键关系，依赖字符串逻辑关联。
- 用户拥有角色/装饰使用 CSV 字符串字段：
 - 查询和约束能力弱。
 - 容易产生重复或无效 slug。
- 商城 `targetId` 没有外键校验；购买时只按字符串写入拥有列表，是否存在目标角色/装饰待进一步校验。
- 前端 `src/main.jsx` 过大，包含所有页面组件、API helper、回放、音频、后台表单等，后续维护成本会升高。
- 全局样式集中在 `src/styles.css`，规模较大，组件样式边界不清。
- 技能扩展当前由 `effectType` 分支实现，`paramsJson` 已保留但核心规则尚未通用化。
- 后台审计覆盖了用户和角色的主要操作，但装饰、商城商品的增改删当前未看到写审计日志。
- `CharacterSkill.enabled` 字段存在，但当前公开角色序列化没有明确按该字段过滤技能，语义待确认。
- 认证安全能力较基础：
 - 无限流。
 - JWT 无服务端失效表。
 - 默认 `JWT_SECRET` 模板为 `change-me-in-production`，部署前必须更换。
- 上传文件只按 MIME 类型和扩展生成文件名，未看到图片内容解码校验，待确认是否足够。
- 当前测试覆盖较多规则和后台 helper，但前端交互没有专门的自动化测试。

11. 后续扩展建议

- 建立 Prisma migration 流程：
 - 引入 `prisma migrate dev / migrate deploy`。
 - 将现有 SQLite schema 固化为首个迁移。
- 将核心统计结构化：
 - 在 `GameRecord` 增加 `winnerColor`、`resultReason`、双方最终积分变化等字段。
 - 排行榜、棋舍战绩、用户统计改用结构化字段。
- 拆分前端模块：
 - `pages/HomeScreen.jsx`
 - `pages/RoomScreen.jsx`
 - `admin/*`
 - `modals/*`
 - `api/client.js`
 - `audio/*`

- `replay/*`
- 拆分样式：
 - 按页面/组件拆 CSS，或引入明确的 CSS module/组件样式约定。
- 重构拥有关系：
 - 将 `ownedCharacters`、`ownedDecorations` 从 CSV 改为关系表。
 - 为商城购买添加唯一约束，避免重复购买。
- 强化权限和安全：
 - 登录/注册限流。
 - 更严格的上传校验。
 - 管理员操作二次确认/审计扩展。
 - JWT secret 检查，生产环境禁止默认值。
- 实时服务可扩展化：
 - 若需要多实例，需引入共享房间状态或 Socket.IO adapter。
 - 对局状态可考虑周期性持久化，避免服务重启丢局。
- 技能系统演进：
 - 将技能效果从硬编码分支迁移到 registry。
 - 明确定义 `paramsJson` schema。
 - 将系统消息占位符列表集中维护并在后台显示。
- 排行榜扩展：
 - 支持分页和赛季。
 - 支持按胜率、总局数、近期积分等维度排序。
 - 后端返回常用角色次数，前端可展示“使用 N 次”。
- 角色/装饰资源管理：
 - 为上传文件建立记录表，便于清理无引用文件。
 - 装饰在玩家端的实际展示/装备逻辑需要补齐或明确。
- 测试扩展：
 - 增加 API 路由集成测试。
 - 增加前端关键流程测试：登录、匹配、后台角色保存、排行榜弹窗。
 - 增加回放快照兼容性测试。